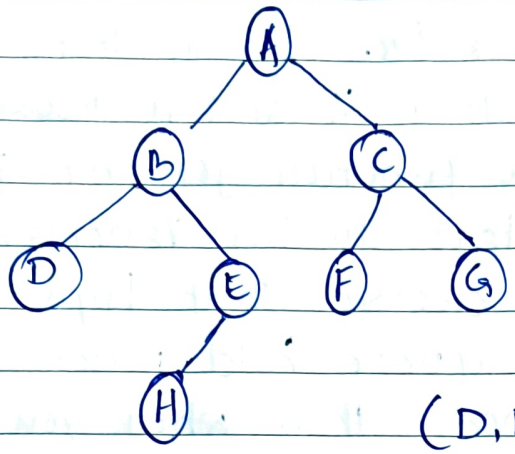
__/__/__

⇒ Lab 2(c): Breadth-First Search

Given this tree, find a path from A to H.



Act: (D, F, G, H) are leaves.

- Write a program that returns the path as a list such as ['A', 'B', 'E', 'H'], and also returns the nodes in the order they were visited.
- BFS and DFS differ by exactly one line. Which line, and why does changing it produce a completely different algorithm?

Ans:

Step 1: Declare the graph as a dictionary

Each key is a node, each value is the list of it's children. Leaves get an empty list []. Do not leave leaves out. If 'D' is missing, then graph['D'] raises keyError the moment the search reaches D.

```
# Lab 2c: Breadth-First Search
graph = {'A': ['B', 'C']
         'B': ['D', 'E']
         'C': ['F', 'G']
         'D': [], 'E': ['H'],
         'F': [], 'G': [], 'H': []}
```

__/__/____

Step 2: Declare the function
def bfs(start, goal) takes where to begin and what to look for. Making them parameters instead of hard-coding 'A' and 'H' means you can reuse the function for any query.

Step 3: Declare the two variables: queue & visited
Here is the decision that trips most students:
does the queue hold nodes or paths?
If it holds nodes, then when you reach H you have no idea how you got there. So we store whole paths. queue = [[start]] is a list containing one list - note the double brackets.
visited is a list of nodes already expanded.

Writing queue = [start] (single brackets) is the classic bug. It looks like it works, then breaks silently.

Step 4 Open the loop and take from the FRONT
while queue: - an empty list is falsy in Python, so the loops ends when nothing is left.
queue.pop(0) removes and returns the item at index 0, the front. That is First In, First out, This single line is what MAKES it BFS.
PATH[-1] is the last element of the list. For ['A', 'B', 'E'] that is 'E' - the node we have currently arrived at. while queue:

```
path = queue.pop(0)
node = path[-1]
```

__/__/____

Step: 5 : Test the goal immediately after popping place this test right after computing node. before anything else.
If you test after expanding, you generate a whole extra layer of nodes before noticing you already had the answer.

```
if node == goal;
    return path, visited
```

Step: 6 Guard against re-expanding
if node not in visited stops a node being expanded twice when two different routes reach it. In a pure tree this cannot happen, but write it anyway: it costs nothing and makes the code correct on any graph.

```
if node not in visited:
    visited.append(node)
```

Step: 7 Expand: build a new path for every child path + [n] creates brand new list. It does not modify path. This matters enormously. If you wrote path.append(n) you would be mutating the same list object for every child, and all of them would end up sharing one corrupted path.
path + [n] = new list, safe ~~path.append(n)~~.
path.append(n) = mutates the shared list, broken.

```
for n in graph[node]:
    queue.append(path + [n])
```

__/__/____

Step : 8 Return failure if the queue empties
If the loop finishes without hitting the goal, no path exists. Return None plus whatever was visited, so the caller can still inspect the search.

return None, visited.

Step : 9 Call it and print
p, v = bfs('A', 'H') unpacks the two returned values into two variables.
The program is complete.

BFS.py

```
# LAB: 2a Breadth-First Search
graph = {'A': ['B', 'C'],
         'B': ['D', 'E'],
         'C': ['F', 'G'],
         'D': [], 'E': ['H']
         'F': [], 'G': [], 'H': []}
def bfs(start, goal):
    queue = [[start]] . # a list of paths not nodes
    visited = []

    while queue:                                    take from
        path = queue.pop(0)  # <--- FIFO: front
        node = path[-1]      # current node = last item
                                                    of path

    if node == goal:
        return path, visited
```

```
        if node not in visited:
            visited.append(node)
            for n in graph[node]:
                queue.append(path + [n])

    return Node, visited
p,v = bfs('A', 'H')
print("BFS path :", p)
print("BFS visited:", v)
```

[output]

```
BFS path : ['A', 'B', 'E', 'H']
BFS visited: ['A', 'B', 'C', 'D', 'E', 'F', 'G']
```